



LECTURE 02

Introduction to Database Systems

The Relational Model and Relational Algebra

Xiaofan Li

School of Computer Science, Nanjing University



Learning Objectives

01

Describe relations as sets of distinct, unordered tuples over attributes and domains, and distinguish schemas from instances

02

Classify key types and explain how primary and foreign keys establish identity and references

03

Compare procedural and declarative query languages; explain closure and the expressive limits of relational algebra

04

Apply and compose selection, projection, set, product, rename, join, and aggregate operations



Lecture Outline

PART I

Relational structure

- Relations, attributes, and domains
- Schemas, instances, and relation properties
- Keys and relationships between relations

PART II

Relational querying

- Query-language paradigms and families
- Selection, projection, and set operations
- Product, rename, composition, and joins
- Aggregation, closure, and operator map



Example of a Relation

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Attributes

Columns

Tuples

Rows



Relations, Attributes, and Domains



Domains, Atomic Values, and NULL

Students Student ID, First Name, Last Name, DoB, Passport No., National ID, Department

An attribute's **domain** is its set of **allowed values**.

Student ID	Alphanumeric string
First / Last Name	Character string
DoB	Date
Passport No.	Alphanumeric; optional
National ID	alphanumeric identifier
Department	Character string

Values are normally **atomic**: treated as **indivisible**.

SQL NULL marks a missing or unknown value where permitted.

NULL complicates operations.



Attribute Values and Their Domains

Students Student ID, First Name, Last Name, DoB, Passport No., National ID, Department

Domains

Student ID: alphanumeric string

First / Last Name: character string

DoB: date

Passport No.: alphanumeric, nullable

National ID: alphanumeric identifier

Department: character string

Student ID	First Name	Last Name	DoB	Passport No.	National ID	Department
2024CS001	Alice	Chen	12-May-2005	P1234567	NID-200001	Computer
2024EE002	Brian	Lee	03-Oct-2004	NULL	NID-200002	Electrical



Schemas, Instances, and Relation Properties



Relation Schema and Instance

Attributes: $A_1, A_2, \dots, A_n$ Schema: $R = (A_1, A_2, \dots, A_n)$

Example: *instructor* = (ID, name, dept_name, salary)

$$r \subseteq D_1 \times D_2 \times \dots \times D_n$$

A relation instance is a set of tuples $t = (a_1, \dots, a_n)$, where $a_i \in D_i$.



Unordered Relations, Unique Tuples

The **order of rows** is irrelevant.

No two tuples may be identical.

Reordering the rows gives the same relation instance.

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Keys and Relationships



Superkeys, Candidate Keys, and Primary Keys

Let $K \subseteq R$, where R is the set of attributes. Example: *instructor*(ID, name, dept_name, salary).

Superkey

Values of K **uniquely** identify a tuple in every legal instance.

$\{ID\}$ and $\{ID, name\}$ are superkeys of *instructor*.

Candidate key

A minimal superkey: no proper subset is a superkey.

$\{ID\}$ is a candidate key.

Primary key

One selected candidate key.

Surrogate / synthetic key

A system-generated identifier created only to distinguish rows, such as an auto-incremented ID or UUID. It carries no required business meaning.



Choosing a Student Key

Students(student_id, first_name, last_name, DoB, passport_no, national_id, department)

Superkeys {student_id} and {student_id, DoB}

Candidate keys Student ID; National ID only if it is mandatory and unique. Names are not guaranteed unique, and Passport No. is optional.

Selected primary key: Student ID

Why select Student ID? It is issued and controlled by the university, always available to enrolled students, stable for university records, and avoids exposing a government identifier. Most importantly, it has **meaning**.

2024CS001

2024-CS-001

2024 Entry year

CS Department

001 Sequence number



Alternate, Simple, and Composite Keys

Candidate keys may be simple or composite. These words describe their number of attributes; they can also describe primary, alternate, or foreign keys.

Alternate key A candidate key not selected as the primary key; e.g., National ID under the stated assumptions.

Simple candidate key

One attribute, such as Student ID.

Composite candidate key

Multiple attributes; a name pair qualifies only if uniqueness is guaranteed.

Student ID	First Name	Last Name	DoB	Passport No.	National ID	Department
2024CS001	Alice	Chen	12-May-2005	P1234567	NID-200001	Computer
2024EE002	Brian	Lee	03-Oct-2004	NULL	NID-200002	Electrical
2024EC003	Carol	Davis	21-Jan-2005	G2345678	NID-200003	Electronics
2024CE004	Daniel	Kim	06-Jun-2004	NULL	NID-200004	Civil
2024CS005	Emma	Garcia	18-Dec-2005	X3456789	NID-200005	Computer



Foreign Keys

A foreign key belongs to the **referencing relation** and matches a candidate key (normally the primary key) of the **referenced relation**. The foreign key itself need not be unique.



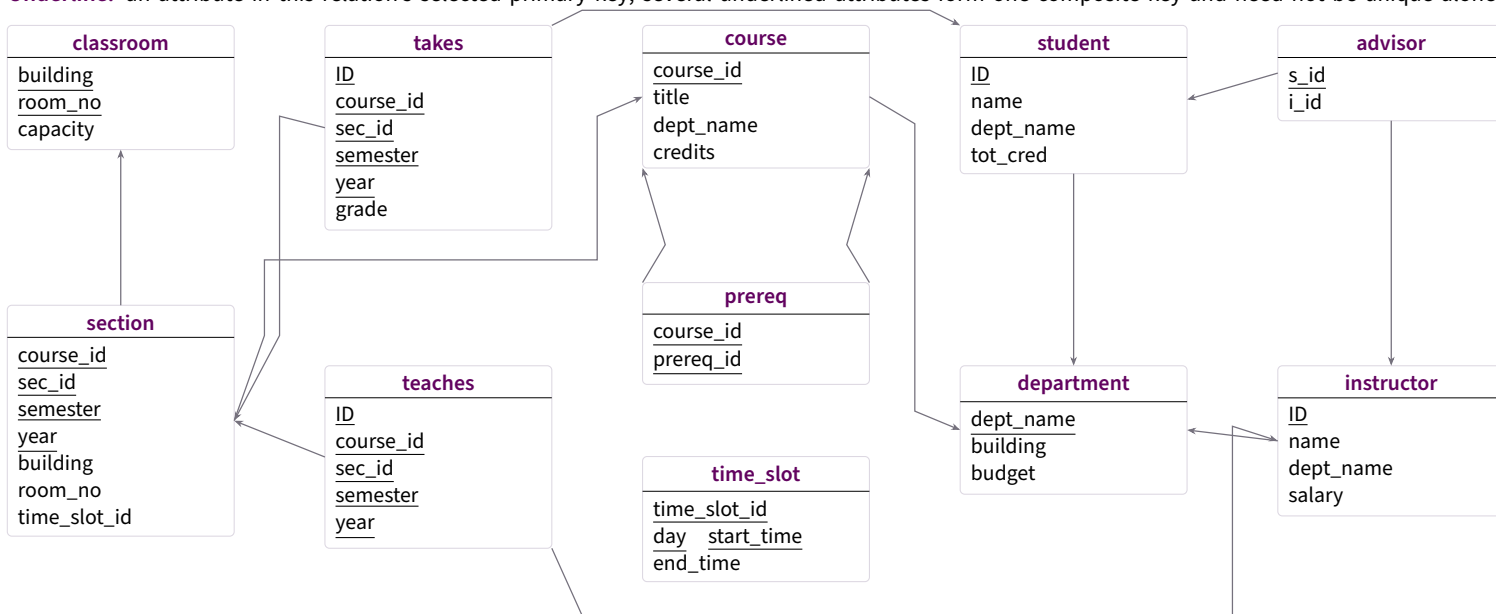
Enrolment.student_id → *Students.student_id* and *Enrolment.course_id* → *Courses.course_id*.

Student ID alone is **not** a superkey of Enrolment because one student may take several courses. Under the one-enrolment-per-course assumption, {student_id, course_id} is a composite candidate key of Enrolment.



University Database Schema

Underline: an attribute in this relation's selected primary key; several underlined attributes form one composite key and need not be unique alone.



Relational Query Languages



Procedural and Declarative Languages

Procedural

Specify **how** to obtain the result.

The programmer supplies an appropriate algorithm for the required inputs.

Declarative

Describe **what** result is wanted.

Specify the relationships that must hold among the values.



Procedural and Declarative Specifications

For $n > 0$ and tolerance $\delta > 0$:

Procedural

Choose $x_0 > 0$; set $i = 0$.

Compute $x_{i+1} = \frac{x_i + n/x_i}{2}$.

If $|x_{i+1} - x_i| \leq \delta$, stop.

Otherwise increment i and repeat.

Declarative

Find $m \geq 0$ such that

$$m^2 = n.$$



Families of Relational Query Languages

- Three “pure” languages
 - Relational algebra
 - Tuple relational calculus
 - Domain relational calculus
- Equivalent expressive power for relational algebra and safe relational calculus
- We will concentrate on relational algebra
 - Not Turing complete: it cannot express every algorithm
 - Six basic operations

Relational Algebra Operators



Basic Properties of Relations

A relation is a **set**: tuple order is inconsequential, and tuples are distinct.

<i>A</i>	<i>B</i>
a_1	b_1
a_1	b_2
a_2	b_1
a_2	b_2

same relation

<i>A</i>	<i>B</i>
a_1	b_1
a_2	b_1
a_2	b_2
a_1	b_2

<i>A</i>	<i>B</i>
a_1	b_1
a_1	b_2
a_1	b_2
a_1	b_1

Repeated tuples:
not a relation

<i>A</i>	<i>B</i>
a_1	b_1
a_1	b_2

Distinct tuples:
valid relation



Selection: Choosing Rows

Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10



$$\sigma_{A=B \wedge D > 5}(r)$$

A	B	C	D
α	α	1	7
β	β	23	10

If AND becomes OR: $\sigma_{A=B \vee D > 5}(r) = r$.



Projection: Choosing Columns

Relation r

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

$\xrightarrow{\pi_{A,C}}$

A	C
α	1
α	1
β	1
β	2

$\longrightarrow$

A	C
α	1
β	1
β	2

$\pi_{A,B}(r)$ retains four distinct tuples.

Remove duplicate tuples.

$\pi_A(r) = \{(\alpha), (\beta)\}$.



Union

 r

A	B
α	1
α	2
β	1

 s

A	B
α	2
β	3

 $\cup$ $=$ $r \cup s$

A	B
α	1
α	2
β	1
β	3

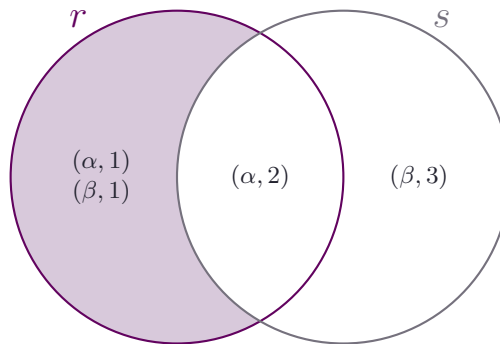


Set Difference

For union-compatible relations $r(A, B)$ and $s(A, B)$, $r - s$ contains the tuples that are in r but not in s .

r	
A	B
α	1
α	2
β	1

s	
A	B
α	2
β	3



$r - s$	
A	B
α	1
β	1

The shaded region is $r \setminus s$: shared tuples are removed.

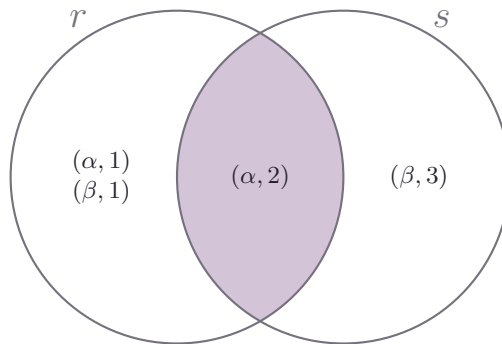


Set Intersection

For union-compatible relations $r(A, B)$ and $s(A, B)$, $r \cap s$ contains exactly the tuples common to both relations.

r	
A	B
α	1
α	2
β	1

s	
A	B
α	2
β	3



$r \cap s$	
A	B
α	2

$r \cap s = r - (r - s)$. The shaded overlap contains the tuples present in both inputs.



Cartesian Product

$$r$$

A	B
-----	-----

α	1
----------	---

β	2
---------	---

$$s$$

C	D	E
-----	-----	-----

α	10	a
----------	----	---

β	10	a
---------	----	---

β	20	b
---------	----	---

γ	10	b
----------	----	---

$$r \times s$$

A	B	C	D	E
-----	-----	-----	-----	-----

α	1	α	10	a
----------	---	----------	----	---

α	1	β	10	a
----------	---	---------	----	---

α	1	β	20	b
----------	---	---------	----	---

α	1	γ	10	b
----------	---	----------	----	---

β	2	α	10	a
---------	---	----------	----	---

β	2	β	10	a
---------	---	---------	----	---

β	2	β	20	b
---------	---	---------	----	---

β	2	γ	10	b
---------	---	----------	----	---

$$\pi_{A,B}(r \times s) = r \quad \pi_{C,D,E}(r \times s) = s$$



Attribute Names in Cartesian Products

$$r$$

A	B
-----	-----

 α 1

 β 2

$$w$$

B	D	E
-----	-----	-----

 α 10 a

 β 10 a

 β 20 b

 γ 10 b

$$r \times w$$

A	$r.B$	$w.B$	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b



Rename

$\rho_w(r)$ returns the same tuples and attributes as r , but gives that relation a new name, w .

Why does r become w ? In a self-product we need two distinguishable copies. The rename changes only the **relation alias**; it does not change the data.

$$r$$

A	B
α	1
β	2

$$r \times \rho_w(r)$$

$r.A$	$r.B$	$w.A$	$w.B$
α	1	α	1
α	1	β	2
β	2	α	1
β	2	β	2



Composition of Operations

For $r(A, B)$ and $s(C, D, E)$, build expressions by combining operations: $\sigma_{A=C}(r \times s)$.

 $r \times s$

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

 $\sigma_{A=C}(r \times s)$

A	B	C	D	E
α	1	α	10	a
β	2	β	10	a
β	2	β	20	b



Natural Join

Let r and s have schemas R and S . Their **natural join** is a relation on schema $R \cup S$.

- Consider every pair of tuples $t_r \in r$ and $t_s \in s$.
- Retain a pair if the tuples **agree** on every **shared attribute** in $R \cap S$.
- Form one result tuple by keeping all values from t_r and then adding the values of t_s only for attributes in $S \setminus R$. Shared attributes appear once because the two tuples already agree on them.
- Equivalently, the joined tuple t satisfies $t[R] = t_r$ and $t[S] = t_s$.



Natural Join Example

$$r$$

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

$$s$$

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

$$r \bowtie s$$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

$$\pi_{A,r.B,C,r.D,E}(\sigma_{r.B=s.B \wedge r.D=s.D}(r \times s))$$

Aggregation



Aggregate Operators

Can we compute...

SUM

AVG

MAX

MIN

r	
A	B
a	5
b	2
c	3

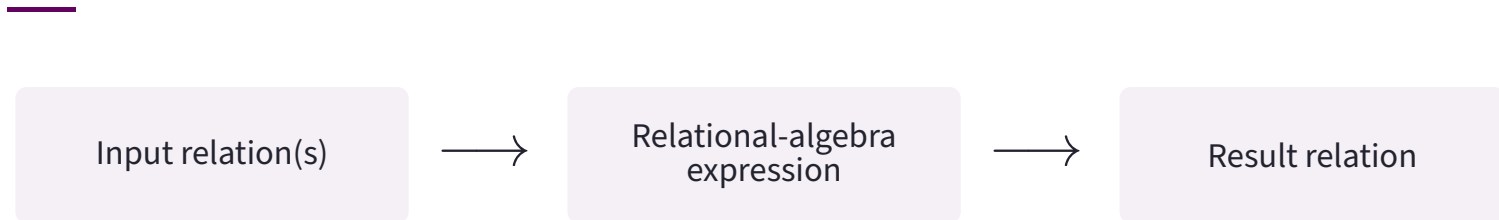
$$\text{SUM}(B) = 10$$

$$\text{MAX}(B) = 5$$

$$\text{SUM}_B(\sigma_{B>2}(r)) = 5 + 3 = 8$$



Closure and Limits of Relational Algebra



Closure: every operator returns a relation, so its result can become the input to another operator.

Turing complete

A language can express every computable algorithm, given enough time and storage.

Why classical algebra is not

Its expression has a fixed, finite number of operators and no data-dependent loop or recursion. It cannot compute arbitrary-length reachability (transitive closure).



Relational Algebra Operator Map

Basic

Selection σ Projection π Union $\cup$

Difference $-$ Cartesian product $\times$ Rename ρ

Derived

Why derived? They can be rewritten using only basic operators, so they add convenient notation but no expressive power.

$$r \cap s = r - (r - s)$$

$$r \bowtie s = \pi_{R \cup S} \left(\sigma_{\bigwedge_{A \in R \cap S} r.A = s.A} (r \times s) \right), \text{ with each shared attribute kept once.}$$

Extended

Aggregation: SUM, AVG, MIN, MAX



Lecture Summary

01

A relation is a set of distinct, unordered tuples; its schema defines structure and its instance contains current data

02

Candidate keys minimally identify tuples; primary and foreign keys establish identity and references

03

Relational algebra is procedural because an expression composes operations that construct the result; relational calculus instead states the condition that result tuples must satisfy

04

Algebra is closed under composition; basic operators support derived joins and intersection, while aggregation extends it